Indic Voice Agent

Directory and code review

Review date: 13 October 2026

Workspace: /home/sinan/indic-voice-agent

Scope: the current files on disk, including uncommitted changes, local translation checkpoints, and the nested Suno Sutra reference repository. Main repository base commit: d2321c8. Reference repository base commit: eedb2bc.

This report explains the architecture, each directory, the responsible functions and code, operational behavior, verification results, and material limitations. Paths are relative to the workspace unless stated otherwise. Line references describe this snapshot and will shift if the files are edited.

The application source was not changed during this review. Credentials in .env were not read or reproduced. Installed libraries, Git internals, model weights, fonts, and images are described by role rather than reproduced or audited individually. The archived Bhashini server was inspected selectively to establish provenance and explain the reference architecture.

Contents

1. What this project does 3

2. Directory inventory and ownership 4

3. Startup and configuration 5

4. Microphone capture, queue and turn state 6

5. vad/ - isolated microphone and speech checks 7

6. stt/ - transcription and device behavior 8

7. nmt/ - local translation adapter and checkpoints 9

8. nmt/bhashini_engine/ - the translation internals 10

9. llm/ - Groq requests and conversation memory 11

10. tts/ - speech synthesis and playback 12

11. The two pipeline implementations compared 13

12. suno-sutra-sw/ - separate reference system 14

13. Reference provisioning, firmware and model archive 15

14. Verification performed and what remains unverified 16

15. Review findings and recommended priorities 17

16. Practical navigation and debugging guide 18

Appendix A. Responsible code excerpts 19

Appendix B. Translation and successful conversation turns 20

Appendix C. Device placement, speech output and token limits 21

1. What this project does

This is a Linux command-line voice assistant for Indian languages. It captures microphone audio, decides when an utterance has finished, transcribes it, translates the text into English, asks a cloud language model, translates the reply into the selected language, generates speech locally, and plays that speech through the default output device.

The maintained entry point is `pipeline/main_optimized.py`. The selected spoken language is currently Malayalam (`LANGUAGE = "ml"`, line 73). The input device is index 4, with two channels, 16,000 samples per second, and 512-sample blocks (lines 28-47). These are fixed script constants, not runtime command-line options.

The processing flow is:

```
Microphone (16 kHz, float32, 2 channels)
  -> copied blocks in a bounded queue
  -> channel average -> mono
  -> Silero VAD -> completed utterance
  -> IndicConformer STT -> Malayalam text
  -> IndicTrans2 / CTranslate2 -> English text
  -> Groq API / gpt-oss-20b -> English reply
  -> IndicTrans2 / CTranslate2 -> Malayalam reply
  -> Indic Parler-TTS -> float32 waveform
  -> default speaker at the TTS model's sample rate
  -> discard queued audio -> listen again
```

VAD means voice activity detection: it answers whether an audio block contains speech. STT means speech-to-text. NMT means neural machine translation. TTS means text-to-speech. None of these stages is trained by this repository; the code loads pretrained models and coordinates inference.

This is a continuously running, sequential assistant. The microphone callback runs independently, but the main processing loop handles one utterance and one reply at a time. There is no streaming transcription, token-by-token response playback, or barge-in mechanism for interrupting an answer. Audio received while processing and speaking is discarded afterward.

The system is hybrid: speech and translation inference run locally; the LLM runs at Groq. First-time Hugging Face model loading can require downloads. Calling the entire system offline is therefore inaccurate, even though local NMT inference itself does not call a translation service.

The earlier `pipeline/main.py` differs materially: it sends the original transcription directly to Groq and requests a reply in the user's language. It does not insert NMT stages. The README's general translation flow describes the optimized entry point, not both scripts.

2. Directory inventory and ownership

Approximate disk usage from this snapshot: 12 GiB total. These figures are rounded filesystem allocation from `du`, not precise model-memory requirements.

Section	Contents and responsibility	Approximate size
pipeline/	Two assembled voice loops; orchestration, audio queue, utterance state, synthesis and playback helpers	100 KiB
vad/	Four microphone/VAD experiments	20 KiB
stt/	Two IndicConformer loading and transcription experiments	12 KiB
nmt/	Translation adapter, vendored inference engine, smoke check, two model checkpoints	428 MiB
tts/	Standalone Malayalam speech generation and playback experiment	8 KiB
llm/	Shared Groq conversation client, live API check, offline contract tests	36 KiB
suno-sutra-sw/	Separate hardware-oriented reference repository, deployment assets, model archives and Git history	4.5 GiB
.venv/	Python 3.12 virtual environment and installed dependencies	7.0 GiB
.git/	Main repository history and metadata	620 KiB
.vscode/	Local editor environment setting	8 KiB
.env	Local credentials/configuration file; ignored by Git	Small
.agents/, .codex/, .aws/	Environment-provided directories; no project files appeared in the inventory	Negligible

Most disk space belongs to installed ML dependencies and reference assets, rather than application code. The nested reference repository has about 2.3 GiB of Git storage and 2.2 GiB under rootfs/. The active translation checkpoints occupy about 428 MiB. Removing the reference repository would not change the adapter's default checkpoint paths, but it would remove the documented local source for reconstructing them.

Root files:

- `README.md`: setup, model acquisition, entry points, language configuration and hardware notes. Useful operational documentation, but some comments/messages inside the code are stale.
- `requirements.txt`: main environment dependencies. Most speech/LLM dependencies have exact versions; the NMT support packages are unpinned. Parler-TTS is installed from a specific Git commit.
- `notes.txt`: informal explanation of microphone, sound card, ALSA, PortAudio, sounddevice and channels. The audio stack is illustrative; actual PortAudio host routing can differ and need not pass through every listed sound server.
- `.gitignore`: excludes the virtual environment, editor settings, bytecode, .env, WAV files, local checkpoints and the nested Suno Sutra repository.
- `.vscode/settings.json`: sets the Python environment manager to VS Code's venv manager; it does not select audio devices or model configuration.

The main checkout already had modified README, pipeline scripts, LLM smoke check and requirements, plus untracked shared LLM client/tests. This report follows those current files, rather than assuming HEAD contains the whole working system. Generated __pycache__ directories are import caches, not independent application layers.

3. Startup and configuration

Responsible code: `pipeline/main_optimized.py:1-306`, `llm/llm_client.py:16-28`, and `nmt/nmt_client.py:68-147`.

The optimized script sets `PYTORCH_CUDA_ALLOC_CONF` to `expandable_segments:True` only if the environment has not already set it. This happens before importing torch (lines 1-12). It adjusts PyTorch CUDA allocation behavior; it does not control ONNX Runtime or CTranslate2 memory pools.

It adds the repository root to sys.path (line 22) so direct execution of a script inside pipeline/ can import the sibling llm/ and nmt/ directories. The same pattern appears in the LLM and NMT standalone checks.

Setting	Current value / source	Effect
SAMPLE_RATE / CHUNK_SIZE	16000 / 512	Each microphone block represents 32 ms
DEVICE / CHANNELS	4 / 2	Specific microphone index and stereo input
VAD_THRESHOLD	0.5	A block at or above this probability counts as speech
SPEECH_CHUNKS_REQUIRED	3	Require nominally 96 ms of consecutive speech
SILENCE_CHUNKS_REQUIRED	10	Require nominally 320 ms of consecutive silence
LANGUAGE / DECODING	ml / ctc	Fixed STT language and CTC decoding
LLM_LANGUAGE	en	NMT bridge language and expected Groq reply language
STT_DEVICE / VAD_DEVICE	cpu / cpu	Preserve GPU capacity for synthesis
TTS_DEVICE	cuda if available, otherwise cpu	Select synthesis device automatically
GROQ_API_KEY	Environment, then project .env	Required cloud API credential
NMT_DEVICE	Environment, default cpu	CTranslate2 device
NMT_*_CHECKPOINT	Environment, otherwise nmt/checkpoints/	Two translation model paths

Startup order is VAD, STT, Groq client, both NMT models and warm-ups, then TTS and tokenizers, then microphone stream. Models are reused across turns, avoiding repeated cold starts. `LANGUAGE_NAMES` validates the selected short language code, but this validation does not prove every stage supports that language equally well.

The shared Groq client loads .env using a path anchored to the repository. Existing environment variables take precedence. In the optimized pipeline this occurs before `IndicTransTranslator.from_env()`, so NMT settings in .env can also become visible there. The standalone NMT check does not load .env itself; it reads only variables already in the process environment.

Missing Groq credentials raise ValueError. NMT load failure expressed as NMTError produces a warning and disables translation. Missing Python imports can fail earlier, before that NMT fallback is reached, because nmt_client imports the engine and its dependencies at module import time.

Both pipeline scripts perform startup at module scope and lack a main guard. Importing either module can load models, initialize credentials and start audio. They should be treated as executable scripts rather than safe library modules.

4. Microphone capture, queue and turn state

Responsible code: `pipeline/main_optimized.py:410-552` and `647-670`.

The microphone stream requests float32 blocks shaped `(512, 2)`. `audio_callback()` copies each block because the audio library's input buffer is temporary. It uses `put_nowait()` into `queue.Queue(maxsize=100)` so slow processing does not block the audio thread. A full queue silently drops the arriving block; existing older blocks remain queued.

The queue can hold about 3.2 seconds of audio: 100 blocks times 32 ms. At the current settings its sample payload is about 400 KiB, excluding Python and NumPy overhead. This bounds backlog, but not the growing list of chunks for an active utterance.

The main loop waits on `audio_queue.get()`, averages input channels with `chunk.mean(axis=1)`, converts the mono waveform to a float tensor, and passes it to Silero. VAD inference runs on the main loop, not inside the optimized script's callback.

The state machine uses `speech_active`, `speech_count`, `silence_count`, and `utterance_chunks`:

1. While idle, each speech block increments speech_count and clears silence_count. Each non-speech block clears speech_count and increments silence_count.
2. The third consecutive speech block starts the utterance. The buffer is initialized to only that current block, then the loop continues.
3. While active, every subsequent block is appended, including silence. A new speech block resets the silence counter.
4. Ten consecutive silent blocks end the utterance. All buffered stereo blocks are concatenated and averaged into mono.
5. The waveform becomes a `(1, number_of_samples)` tensor on `stt_model.d`, then enters the full response pipeline.
6. After playback, queued audio is drained and counters/buffers are cleared. The next loop iteration resumes listening.

Important consequences:

- The first two confirming speech blocks are not saved, so approximately 64 ms of detected onset is omitted. Speech below the initial threshold is also absent. There is no pre-roll ring buffer.
- Ten silent blocks remain in the STT input, adding about 320 ms of trailing silence. A pause this long splits an utterance, even if the speaker intends to continue.
- The 96 ms / 320 ms figures assume uninterrupted blocks. Queue drops or audio overflow break that wall-clock assumption.
- Long speech or persistent false-positive speech can grow utterance_chunks without a maximum duration. The queue's size limit does not prevent this.
- Audio captured while transcribing, waiting for Groq, generating speech, or playing a reply is eventually discarded. The assistant cannot accept an interruption during that interval.
- The code resets its counters, but does not explicitly reset Silero's internal recurrent state after skipped audio. The effect on the next turn requires a dedicated audio test.

`play_audio()` waits for playback. `clear_audio_queue()` then drains until empty, reducing replay of the assistant's own voice. This is not acoustic echo cancellation: live input after the drain can still contain room reverberation or other sounds.

5. vad/ - isolated microphone and speech checks

These scripts are experiments and manual smoke checks. Their names do not make them assertion-based automated tests. They execute at module scope, so indiscriminate test discovery/import can trigger hardware access or model loading.

File	Responsible code	What it does
mic_test.py	lines 8-28	Reports default input device, records five seconds at 16 kHz mono float32, then reports sample count, min, max, peak and RMS
vad_test.py	lines 12-63	Loads Silero, records five seconds, detects speech timestamps for the entire recording, converts sample indices to seconds
realtime_vad.py	callback, lines 37-67; stream, 73-86	Runs Silero on each mono block inside the callback and prints start/end transitions at probability > 0.5
test_vad.py	lines 4-13	Prints torch/CUDA details and loads Silero on CUDA if available, otherwise CPU

`mic_test.py` isolates basic capture. A nearly zero RMS can indicate silence, a wrong source or muted capture; it is not proof of any one cause. Peak and RMS are diagnostic values rather than pass/fail assertions.

`vad_test.py` uses Silero's `get_speech_timestamps()` helper on a complete recording. It is different from the integrated pipeline's custom consecutive-block counters. Reported timestamp boundaries therefore need not match the pipeline's segmentation.

`realtime_vad.py` flips state on a single threshold crossing, with no three-block start confirmation or ten-block silence wait. Its threshold is strict `> 0.5`, while the optimized pipeline uses `>= 0.5`. It runs inference and printing inside the audio callback, which can miss callback deadlines on a slow device. The integrated pipeline's queued processing is the more appropriate ownership split for substantial inference.

The standalone scripts generally use the default input device and one channel, whereas the integrated entry point uses explicit device 4 and two channels. A successful standalone microphone check does not automatically prove the integrated settings are correct.

Data contracts: VAD needs a one-dimensional audio waveform at the declared sampling rate, and each integrated block is 512 samples at 16 kHz. Channel averaging is a downmix, not resampling. The pipeline assumes the audio backend can supply the requested rate; it has no explicit application-level resampler.

6. stt/ - transcription and device behavior

Responsible integration code: `pipeline/main_optimized.py:186-211` and `552-575`. Standalone code: `stt/stt_test.py:24-98` and `stt/test_stt_model_load.py:8-14`.

The model ID is `ai4bharat/indic-conformer-600m-multilingual`. `AutoModel.from_pretrained(..., trust_remote_code=True)` loads custom model code, rather than a standard Transformers-only implementation. The completed waveform is sent to `stt_model(wav, LANGUAGE, DECODING)` inside `torch.inference_mode()`. The selected language is provided explicitly; the pipeline does not detect it from speech.

CTC is connectionist temporal classification, a decoding approach for sequence transcription. Here `"ctc"` is passed to the model's custom call interface; the repository does not implement the decoder itself.

The optimized loader contains a specific CPU-placement workaround. It saves `torch.cuda.is_available`, temporarily replaces it with a function returning False when STT_DEVICE is cpu, constructs the model, and restores the original function in a finally block. According to the source comments, the remote implementation chooses ONNX Runtime sessions using its own CUDA availability check; calling `.to("cpu")` alone does not relocate those sessions.

This workaround is significant because ONNX Runtime allocations are outside PyTorch's allocator. Changing PyTorch's allocation setting or emptying its cache cannot directly release ONNX session memory. `stt_model.d` is used for the input waveform because the remote model owns the actual device convention.

The temporary patch is process-global. Current startup is sequential, and restoration is protected even if construction fails. Concurrent model initialization would require revisiting this technique. The remote model's exact current implementation and actual provider placement were not loaded during this review, so comments about its approximately 27 sessions are recorded as the source's rationale, not freshly measured hardware facts.

`stt_test.py` records five seconds of Malayalam audio using device 4 and one channel, writes `test.wav` in the current working directory, builds a batched tensor on model.d, and prints a CTC transcription. It does not include the optimized CPU workaround. `test_stt_model_load.py` loads the model and prints CUDA availability; contrary to the README's description, it does not print model.d itself.

There is no guard that skips an empty transcription before translation and Groq. STT errors are not caught per turn. Model loading can download remote artifacts, and the revision is not pinned in the pipeline. This makes cached/local model versions part of reproducibility and establishes a trusted-code boundary.

7. nmt/ - local translation adapter and checkpoints

Responsible code: `nmt/nmt_client.py`, particularly the language map at 42-66, loader at 81-95, constructor at 101-126, environment factory at 128-147, direction selector at 149-158 and translation methods at 167-206.

`IndicTransTranslator` owns the application-facing translation contract. It loads one Indic-to-English model and one English-to-Indic model once, using CTranslate2. Default inference is on CPU with beam width 5 and maximum decoding length 256. The adapter uses direct Python calls; it does not start the Suno Sutra Flask server or any systemd service.

Checkpoint locations:

```
nmt/checkpoints/indic-en/ct2_int8_model/
nmt/checkpoints/en-indic/ct2_int8_model/
```

Each directory contains config.json, model.bin, source_vocabulary.txt, target_vocabulary.txt and vocab/ SentencePiece models. The two model.bin files are 214,677,593 and 214,673,465 bytes respectively. These are int8 checkpoints; file size does not equal peak RAM usage. CTranslate2 owns execution of the trained weights.

The adapter maps short language codes to language-and-script identifiers. Examples: `hi -> hin_Deva`, `ml -> mal_Mlym`, `ta -> tam_Taml`, `en -> eng_Latn`. The map includes 22 Indic languages plus English. Kashmiri defaults to Arabic script, Manipuri to Bengali script and Sindhi to Arabic script. These choices must match the transcription's script for best results.

`_direction()` permits only translation into English or out of English. Direct Hindi-to-Malayalam translation is unsupported by this wrapper, even though the reference archive has a broader routing implementation.

`translate()` returns None, empty/whitespace strings and same-language input unchanged. For nontrivial input it validates both language codes, selects a direction and calls `Model.translate_paragraph()`. Internal exceptions from that path are wrapped as NMTError, allowing the caller to implement fallback. The initial `text.strip()` happens before the try block, so the documented wrapping is not an all-input type-validation guarantee; the intended contract is text strings or None.

`warm_up()` runs a tiny sample in each configured direction. For any non-English source it uses the Hindi word for a greeting, even when the source is labelled Malayalam or another language. This exercises the inference path but is not a language-correct quality check.

The optimized pipeline's `translate_text()` returns `(text, elapsed_seconds)`. It skips translation when the languages match or no translator loaded. NMTError causes a warning and returns the original text. This preserves availability at the cost of the language guarantee: the English-only LLM prompt remains active, so an unavailable reverse translation can lead to English speech.

Startup diagnostics at optimized lines 235-243 incorrectly mention an HF cache and network for these local NMT checkpoints. The actual adapter paths and README acquisition instructions point to local CTranslate2 directories extracted from bhashini_models.zip.

8. nmt/bhashini_engine/ - the translation internals

This engine is vendored from Suno Sutra's Bhashini archive, itself based on AI4Bharat IndicTrans2. It supplies the linguistic preprocessing and postprocessing needed around neural translation; simply passing raw text directly into model.bin would omit those steps.

The active call chain is:

```
IndicTransTranslator._translate()
  -> Model.translate_paragraph()
  -> split_sentences()
  -> Model.batch_translate()
  -> preprocess_batch(): normalize -> script handling -> SentencePiece
  -> split long token sequences -> add source/target language tags
  -> ctranslate2_translate_lines() -> best generated hypothesis
  -> postprocess(): join subwords -> restore entities -> native script
  -> join sentences into a paragraph
```

`engine.py:38-60` splits English using both Moses and NLTK and chooses the output with fewer sentences. Indic input uses indic-nlp sentence splitting. English translation therefore requires NLTK tokenizer resources as well as the Python package; requirements.txt does not install the data files. This snapshot had the needed resources because the real English-to-Hindi check passed.

`Model.__init__()` at lines 139-200 creates Moses helpers, an Indic transliterator, source and target SentencePiece processors, and a CTranslate2 Translator. The adapter passes `model_type="ctranslate2"`, so that is the only active backend.

`preprocess_sent()` at 366-412 normalizes punctuation and protected spans, tokenizes text and converts many Indic scripts to a common Devanagari representation. Arabic, Aran, Ol Chiki, Meitei and Latin script inputs are exempted from that input transliteration. `apply_spm()` at 354-364 converts normalized text into SentencePiece subword tokens. `apply_lang_tags()` at 80-97 prefixes source and target identifiers.

`ctranslate2_translate_lines()` at 202-218 performs batch inference with maximum batch size 32, beam width 5, output limit 256, and `max_input_length=160`. It chooses the first hypothesis per item. Preprocessing splits sequences into groups of up to 256 tokens before adding language tags (100-131 and 348-351). The 160-token backend limit is lower than that split size, so long sentences can be truncated before translation. This is a correctness concern for lengthy utterances or replies, not demonstrated by the short smoke samples.

`postprocess()` at 441-505 reconstructs words from SentencePiece markers, normalizes Perso-Arabic punctuation, restores protected entities, detokenizes and transliterates into target scripts. It includes an Odia correction. The source and target SentencePiece models are loaded, but the active output path manually removes subword spacing rather than calling the target processor's decode method.

Other files:

- `flores_codes_map_indic.py`: maps FLORES language/script identifiers to indic-nlp codes and ISO inputs to FLORES identifiers; several languages share a normalizer code.
- `normalize_punctuation.py:14-59`: punctuation/spacing normalization, ported from a Perl utility. Its header explicitly warns against directly editing this vendored file.
- `indic_num_map.py:6-117`: lookup table converting native numeral characters into ASCII digits. This aims to preserve numeric value, not original glyph styling.
- `normalize_regex_inference.py:8-13, 29-105`: protects emails, URLs, numeric expressions, hashtags and handles using placeholders such as `<ID1>`, then records original values for restoration.
- `__init__.py`: empty package marker.

The engine still includes a fairseq branch at 190-198 and a forwarding method at 220-221, despite its docstring saying the fairseq path was dropped. The imported custom_interactive module is absent. It is unreachable through the current adapter, but would fail if that backend were selected directly.

9. llm/ - Groq requests and conversation memory

Responsible code: `llm/llm_client.py:13-40`. Default model: `openai/gpt-oss-20b`. API endpoint: `https://api.groq.com/openai/v1`.

`GroqChat` centralizes credentials, request configuration and conversation history for both pipelines and the live LLM check. It uses the OpenAI-compatible client against Groq's endpoint. The SDK namespace and model prefix do not change the configured provider: the network destination is Groq.

Construction loads the project .env, trims the API key, fails with a setup hint if missing, and creates the client with a 30-second timeout and one retry. History starts with a system message. The timeout is an SDK request setting, not a guaranteed 30-second total turn deadline once retries and other processing are included.

`send_message()` builds a candidate request from committed history plus the new user message. It sends that candidate to `chat.completions.create()`, reads the first choice's message content and rejects missing or whitespace-only output. Only after a successful, nonempty reply does it assign the new history containing both the user and assistant messages.

That success-only commit is a useful invariant: a failed call cannot leave an unmatched user message in history. The offline tests verify both authentication failure and empty-response behavior. The class returns the original response content, not its stripped version.

In the optimized pipeline, successful normal turns store the English translation of user speech and the original English LLM reply. The original Indic transcription and translated spoken reply are printed but not included as separate history records. If input translation fails, a history entry can instead contain untranslated input.

History lives in memory for the current process and is resent in full on every request. There is no persistent session store, conversation reset command, history trimming, token budget, max output token limit or response streaming. Long sessions can grow cost, latency and context usage. A concise-response system prompt influences behavior but does not enforce an output-length cap.

`llm/llm_test.py:11-13` makes a real one-prompt API call without loading speech models. `llm/test_llm_client.py` uses httpx2's MockTransport to inspect requests without contacting Groq. It checks URL and authorization, model selection, follow-up history, failure rollback, empty reply rollback and the missing-key setup hint.

Privacy boundary: raw microphone audio is not supplied to the Groq call by this code. The transcribed/translated user text and prior conversation are supplied, so sensitive spoken content can still leave the machine. Console output also includes transcriptions and replies. No provider retention policy was investigated in this repository review.

10. tts/ - speech synthesis and playback

Responsible integration code: `pipeline/main_optimized.py:288-403` and `619-630`. Standalone experiment: `tts/tts_test.py:23-99`.

The model ID is `ai4bharat/indic-parler-tts-pretrained`. The optimized pipeline loads it once with float16 weights and moves it to CUDA if available, otherwise CPU. It loads one tokenizer for the text to speak and another for the model's voice-description text encoder.

`call_tts(text)` tokenizes the fixed voice description and reply separately. It moves token IDs and attention masks to the selected device and calls `tts_model.generate()` inside inference_mode. The description controls voice qualities such as gender, expressiveness, pace and recording style. The reply supplies the words to pronounce; the pipeline does not pass LANGUAGE as a separate argument to generation.

The generated tensor is moved to CPU, converted to a NumPy array, squeezed into a waveform, and explicitly cast to float32 (line 359). That final conversion matters because sounddevice does not accept float16 playback samples. Float16 model inference and float32 audio output serve different responsibilities.

After converting the result, call_tts deletes generation and token tensors. On CUDA it calls `torch.cuda.empty_cache()`. This can release unused cached allocator blocks, but it does not unload the model, free live tensors or guarantee that every future reply fits in VRAM. The source comments attribute this cleanup to fragmentation on a 6 GiB GPU; no memory profile was collected in this review.

`play_audio()` uses `sd.play(audio, tts_model_sampling_rate)` followed by `sd.wait()`. Playback uses the model's output sampling rate, independently of the 16 kHz microphone rate, and routes to the default output device. There is no explicit output-device selection, normalization, clipping check or application-controlled cancellation.

CPU fallback still requests float16 TTS weights. Compatibility and speed for that path depend on the installed runtime and operators; CPU synthesis was not executed here. The code has no automatic float32 selection for CPU.

`tts_test.py` synthesizes a fixed Malayalam sentence with a friendly female Malayalam voice description, saves `tts_output.wav` in the current working directory and plays it. It loads the model without the optimized script's explicit float16 setting and lacks its per-turn cache cleanup and explicit output float32 cast.

TTS timing in the optimized helper is commented out. The turn summary prints `TTS + gen: see above`, but no active generation-duration or real-time-factor calculation appears above. It reports STT, both translation calls, Groq and playback, rather than a complete measured total or TTS latency.

11. The two pipeline implementations compared

Behavior	pipeline/main.py	pipeline/main_optimized.py
User-to-LLM text	Original transcription	Local NMT to English, with fallback
LLM system prompt	Respond in user's language and native script	Always respond in English
Reply-to-TTS text	Direct LLM response	Local NMT back to user language
Shared Groq client	Yes	Yes
STT requested device	CPU via .to()	CPU plus temporary CUDA-availability patch
TTS model dtype	float16	float16
Waveform dtype conversion	No explicit float32 cast	Explicit float32 cast
CUDA allocation setting	Absent	expandable_segments if not already configured
Per-turn tensor/cache cleanup	Absent	Deletes TTS intermediates; empties CUDA cache
Segmentation and queue structure	Sequential utterance loop	Same basic structure

Both scripts reuse GroqChat rather than duplicating the API implementation. The translation stage is integrated only into the optimized script. That makes the old file a useful historical comparison, but an alternate execution path with different semantics and known device/dtype concerns.

Important ownership boundaries:

- pipeline/: owns turn lifecycle, stage sequencing, audio queue and fallback policy.
- llm/llm_client.py: owns provider requests and commit-on-success history.
- nmt/nmt_client.py: owns application language mapping, checkpoint configuration and translation direction validation.
- nmt/bhashini_engine/: owns linguistic preparation and reconstruction around the model.
- External models/libraries: own learned inference, audio device access and neural generation.

Most integration remains in a single 672-line script. Comments and explicit steps make the dataflow approachable, but module-scope initialization and duplicated lifecycle code make unit-level testing awkward. The reusable LLM and NMT classes are already sensible boundaries for keeping provider and translation logic separate.

There is no web server, UI, REST endpoint, database, packaging manifest, deployment service or application-level scheduler for the main project. Those facilities in suno-sutra-sw belong to the separate reference system and are not activated by running main_optimized.py.

12. suno-sutra-sw/ - separate reference system

This directory is an independent Git repository ignored by the parent project. It targets Suno Sutra hardware, particularly NVIDIA Jetson, and explains where the local NMT engine and checkpoints came from. It is not an imported package of the active pipeline.

Its system differs from the main assistant: a trigger button starts/stops recording, the device also captures a camera frame, and local model services provide recognition, translation, visual-language inference and speech. Hardware UI and service supervision are first-class concerns.

python/ - PocketInfer application layer

`python/setup.py` packages pocketinfer and exposes `pocketinfer-service=pocketinfer.service:main`. `python/pocketinfer.service` launches that command through systemd, uses a Jetson-specific working path under /home/ubuntu, and restarts on exit.

`python/pocketinfer/service.py:19-83` parses application selection, logging, dummy-board files and settings overrides. It resolves an application through ApplicationRegistry, chooses a hardware or dummy board, starts a background memory-display updater, starts the application and waits for it to stop.

`applications/registry.py` registers classes through the RegisterApplication decorator and stores metadata. `applications/base.py:8-38` provides settings, model-manager ownership and an application thread lifecycle. These abstractions support multiple hardware applications, rather than the main repository's single script.

`applications/hear_the_world.py:100-210` implements the multilingual visual assistant. It waits for local services, waits for a button press, records audio and captures a JPEG, stops recording on release, selects Vosk for English or Bhashini for Indic ASR, translates a non-English query, calls Ollama with the image and question, translates the reply if needed, synthesizes through Bhashini and plays the result. It catches errors inside its loop and writes logs/images/audio under /tmp/hear_the_world_en_logs.

`applications/hear_the_world_en.py:42-103` is the simpler English route using Vosk, Ollama and Piper. Its metadata currently selects ministral-3:3B, while the multilingual application's metadata selects qwen3-vl:2b. These are reference-system model choices, not the active project's Groq default.

Hardware and model helpers

- `audio.py:104-197`: PyAudio recording thread, capture-rate probing, 16-bit frames and AudioData conversion; `222-277`: raw PCM playback through ffplay; dummy recorder at 202-218 reads a file.
- `boards/base.py`: camera thread at 23-71, common Board at 73 onward, audio device selection and button events, hardware selection via get_board(), and a DummyBoard for file-backed operation.
- `boards/jetson.py`: Jetson devboard, native handheld UI and USB IO-expander board variants; also a Raspberry Pi AI Hat board declaration. A declaration is not evidence of full deployment support.
- `serialcomms.py:8-92`: USB serial IOInterface, message readers, subscriptions and request/response transactions.
- `ui/handheld.py:18-265`: 320x240 display text and buttons; later classes provide ILI9341 configuration and multiprocessing RPC support. icons.py and bundled fonts support the screen.
- `models/base.py`: model registration/state tracking, swap monitoring, common model contract and systemd service helpers.
- `models/bhashini.py:24-72`: local HTTP calls to /asr, /nmt and /tts at localhost:11400, with base64 audio payloads.
- `models/ollama.py`, `vosk.py`, `piper.py`: wrappers for local visual-language generation, recognition and speech playback.

The Bhashini HTTP calls do not set an explicit timeout. This is a reference-system availability concern and is independent of GroqChat's request timeout in the main project.

13. Reference provisioning, firmware and model archive

rootfs/ - deployment assets

`rootfs/install_all_usb.yml` applies initial, vllm, indic and app roles to the USB-connected target. `update_only_usb.yml` applies only the app role. `inventory.ini` describes the provisioning target; it is deployment configuration, not the voice-agent microphone configuration.

`roles/initial/tasks/main.yml` prepares the Jetson host: identity, GUI/ZRAM changes, swap, JetPack/Docker, CUDA package setup and system dependencies. These tasks change a target system and were read, not executed.

`roles/indic/tasks/main.yml:3-57` installs CTranslate2 native/Python assets. Lines 59-101 extract Bhashini models and establish a Python environment, including numpy<2 and a forced GPU ONNX Runtime installation. Lines 103-126 build Flite and install voices, 128-131 acquire NLTK punkt_tab resources, and 133-146 install/start the Bhashini service. The bundled CTranslate2 wheel targets CPython 3.10 on ARM64, unlike the main project's Python 3.12 environment.

`roles/app/tasks/main.yaml` deploys application code and hardware/system integration. `roles/vllm/tasks/main.yml` is currently empty despite being named in the full playbook. `roles/sdk/tasks/main.yml` is a separate SDK task file outside that default role list. README.md and Jetson_Flash.md document image flashing and provisioning. Binary wheels, .deb files and model ZIPs are deployment payloads, not Python source.

ioexpander/ - microcontroller firmware

`ioexpander/code.py` is CircuitPython firmware for a Seeeduino XIAO RP2350. It manages trigger/buttons, NeoKey LEDs, haptics and an ILI9341 touchscreen. Key functions are check_buttons() at 277, run_led_anim() at 301 and parse_msg() at 323. It exchanges newline-delimited serial messages with the Jetson. settings.toml, icons.py, fonts and sd/ support firmware deployment and display resources.

Its README defines commands such as text updates, LED control and button notifications. This firmware is copied to the microcontroller's CircuitPython storage; it should not be run as a normal Linux Python application.

assets/ and repository metadata

assets/ contains architecture images, flashing screenshots, hardware photographs and troubleshooting illustrations. LICENSE identifies the reference project license. Its own .gitignore, .gitattributes and .git govern a separate history and Git LFS assets.

What the Bhashini archive contains

`rootfs/roles/indic/files/bhashini_models.zip` has 73 members, including ASR, NMT and TTS code, checkpoints, requirements, install instructions and a service file. Selectively reading the archive establishes the main NMT adapter's source lineage without extracting or starting the entire deployment.

The archive's `bhashini_models/main.py:20-61` loads ASR/NMT/TTS engines and exposes Flask routes /asr, /nmt, /tts, /s2s, /health, /memory and /snap. Its CLI defaults to localhost port 11400 (196-209). Its NMT wrapper loads directions from local checkpoint folders on CUDA and supports Indic-to-Indic as well as English bridging (`nmt/infer.py:32-104`).

The active repository reuses only two translation directions and the preprocessing engine, directly in-process on CPU by default. It does not reuse the archived server, Indic-to-Indic model, reference ASR/TTS implementation, or local Ollama inference route.

Selective review also found the archived /s2s implementation uses unqualified run_asr/run_nmt/run_tts names inside a method (main.py:157-179), which would not resolve to the instance methods as written. That route is unused by the active pipeline. This reference review is architectural coverage, not a full audit of every reference module or archive asset.

14. Verification performed and what remains unverified

The review used directory inventory, disk usage, current source reads, line-numbered call tracing, selective archive reads, installed-package metadata and targeted execution. It did not record microphone audio, play sound, call the live Groq API or load the large STT/TTS models.

Check	Result	What it proves
Python AST parsing of 20 project files	Passed	All main project Python sources parse; does not validate imports or inference
.venv/bin/python -m unittest llm.test_llm_client -v	4 tests passed	Real SDK requests against mocked transport meet client/history contracts
.venv/bin/python nmt/nmt_test.py	Passed	Both local models load and short Hindi/English examples translate in this environment
NMT pass-through/error exercises	Observed expected outputs	Same-language/empty text pass through; unsupported source raises NMTError
Installed environment inspection	Python 3.12.3	Confirms current interpreter and selected package versions

The NMT check loaded both models in approximately 0.7 seconds. Three Hindi-to-English samples took approximately 0.05-0.09 seconds each; three English-to-Hindi samples took approximately 0.10-0.14 seconds each. Example output was "My name is Rahul." and "The weather is very good today." These measurements cover short CPU translations in this single run, not a general performance benchmark or multilingual quality evaluation.

The script's "All NMT checks passed" line is a smoke-check status, not a rigorous translation-quality assertion. The fail-soft exercises print their results and accept NMTError without assertions about exact output. The input and output lists are separate fixed samples, so it demonstrates both directions rather than automatically feeding one translation's output into the other.

Selected installed versions matched the pinned requirements: numpy 2.5.3, torch 2.14.0, transformers 4.46.1 and openai 3.16.2. Unpinned components observed included ctranslate2 4.8.2 and nltk 3.10.3. The offline test also directly imports httpx2 2.13.0, which is not listed as a direct requirement. No clean-room dependency installation was attempted.

Not verified: actual microphone index/channel compatibility, STT recognition quality, remote ONNX provider placement, TTS CPU/GPU execution, speaker output, peak RAM/VRAM, end-to-end latency, multilingual accuracy beyond the Hindi samples, live credentials, long-session context exhaustion or cancellation during processing. These limits do not negate the source-level findings; they constrain claims that the complete audio system is currently working on this hardware.

A blanket test-discovery command was intentionally avoided because several test-named modules load models or record audio at import time. The focused unittest module is the clear offline automated test surface.

15. Review findings and recommended priorities

The project has a clear staged design, reuses loaded models, keeps heavy inference outside the integrated audio callback, bounds queued audio, commits LLM history only after successful replies and provides a deliberate translation fallback. It is a practical prototype with several behavior gaps worth addressing before unattended use.

Higher priority

1. Non-NMT failures stop the active loop. Evidence: optimized lines 484-670 catch only KeyboardInterrupt, while STT, ask_llm(), TTS and playback can raise. The finally block stops/closes an already started stream, but a failed turn does not resume listening. Define per-turn recovery and state cleanup at the pipeline lifecycle boundary; keep useful error diagnostics.
2. NMT input limits can lose information. Evidence: engine preprocessing splits at 256 tokens, but backend inference uses max_input_length=160. Align chunking and backend limits, allowing for language tags, and add a meaningful long-input preservation test.
3. Detected speech onset is omitted. Evidence: optimized lines 515-524 save only the third confirming block. Preserve a small pre-roll at the utterance-buffer owner and verify first-syllable capture with deterministic audio/block sequences.
4. Utterance and conversation growth are unbounded. Evidence: active chunks append without a duration limit; GroqChat keeps all successful turns. Establish limits with a clear user-visible policy rather than silently losing content.

Medium priority

5. Language consistency is best-effort. NMT fallback can send untranslated input and produce English speech; fixed LANGUAGE does not detect language changes. Surface the degraded mode clearly. A successful language-map lookup is not evidence of STT/NMT/TTS support parity.
6. Fresh-machine setup has gaps. NLTK sentence data is external to pip dependencies; model checkpoints are ignored by Git; several NMT packages are unpinned. Document/install needed resources explicitly and capture a tested dependency set. Main STT/TTS model revisions are also unpinned.
7. Startup and cleanup boundaries need care. InputStream construction and start occur before the protected try block at lines 459-484. A start failure is outside the cleanup path. The CUDA-availability monkey patch is safe for current sequential loading but global in scope.
8. CPU TTS fallback is not validated. Float16 is requested on both devices. Exercise the CPU path before promising compatibility, and choose dtype where device compatibility belongs if needed.
9. Audio gaps and VAD continuity are opaque. Queue.Full silently drops incoming blocks; counters reset after skipped processing audio, while internal VAD state is retained. Expose drop counts and verify whether an explicit VAD reset is appropriate.

Lower priority / clarity

10. Timing and diagnostics are misleading. TTS timing is disabled, yet the summary points to it. NMT warnings mention HF/network despite local paths. Correct these to match actual behavior.
11. Historical paths invite confusion. main.py omits NMT and optimized safeguards. Label its behavior prominently or consolidate shared lifecycle code when maintaining both becomes necessary.
12. Documentation/backend drift remains in vendored code. The absent fairseq module contradicts the engine docstring's removal claim. Treat the CTranslate2 path as supported and remove or clearly mark unreachable remnants during a future cleanup.

Privacy and trust: retain the ignored credential file convention; avoid logging secrets; explain that text goes to Groq; pin/review remote STT code when reproducibility matters. The reference visual application additionally writes images and audio to /tmp; the main pipeline does not implement that persistence behavior.

No fixes were made as part of this report. The recommendations identify the responsible existing layers and do not imply a need to add a framework, service or database.

16. Practical navigation and debugging guide

For understanding the application, start with the optimized constants, then the audio loop at line 484, then follow the calls into GroqChat, IndicTransTranslator and call_tts(). The dependency graph is smaller than the full directory size suggests.

Symptom / question	Start here	Why
What reaches the cloud?	llm/llm_client.py:30; optimized:583-596	Request construction and translated input
Why does it miss the beginning?	optimized:515-524	Initial buffer excludes earlier confirming blocks
Why does it cut at a pause?	optimized:47, 503-536	Ten silent blocks close an utterance
Why can't I interrupt?	optimized:594-650	Main loop processes/speaks, then drains captured audio
Translation disabled?	nmt/nmt_client.py:81; optimized:234-278	Local paths, load errors and per-turn fallback
English comes out instead of Malayalam?	optimized:124-136, 608-620	English prompt plus reverse-translation fallback
GPU memory issues?	optimized:188-206, 288-291, 373-378	ONNX STT placement and TTS allocation/cleanup
Groq history/request error?	llm/llm_client.py:30-40	Commit-on-success and API settings
Incorrect translation of long text?	engine.py:100-131, 209-215	Split size and backend input limit
Why is this directory so large?	.venv/, suno-sutra-sw/, nmt/checkpoints/	Installed libraries and binary assets dominate

Useful manual commands, in increasing scope:

```
.venv/bin/python -m unittest llm.test_llm_client -v
.venv/bin/python nmt/nmt_test.py
.venv/bin/python -c "import sounddevice as sd; print(sd.query_devices())"
.venv/bin/python vad/mic_test.py
.venv/bin/python vad/vad_test.py
.venv/bin/python stt/stt_test.py
.venv/bin/python tts/tts_test.py
.venv/bin/python llm/llm_test.py
.venv/bin/python pipeline/main_optimized.py
```

Only the first two commands were executed during this review. Audio scripts record/play sound; the live LLM check sends a request; model-loading scripts can download artifacts. Before running the whole pipeline, match DEVICE and CHANNELS to actual hardware and confirm checkpoints and credentials.

STT and TTS experiments write WAV files relative to the process working directory, not necessarily their own source folders. WAV files are ignored by Git, but still remain on disk until removed. The main pipeline keeps its utterance buffer and generated waveform in memory and does not explicitly save WAVs.

Appendix A. Responsible code excerpts

The following excerpts are taken directly from the reviewed source, with original line numbers. Each illustrates a key invariant or data transformation. Surrounding startup and error-handling context is described in the main report.

A1. Capture without blocking the audio thread

Source: pipeline/main_optimized.py:410-425

```
410  audio_queue = queue.Queue(maxsize=100)
```

Callback body:

```
420      if status:
421          print("Audio status:", status)
422      try:
423          audio_queue.put_nowait(indata.copy())
424      except queue.Full:
425          pass
```

The copy protects the queued data from reuse of the audio callback's buffer. The queue bound prevents unbounded backlog, and the Full branch drops incoming data rather than blocking.

A2. Consecutive-block start confirmation

Source: pipeline/main_optimized.py:503-524

After updating the counters:

```
514          if not speech_active:
515              if speech_count >= SPEECH_CHUNKS_REQUIRED:
516
517                  speech_active = True
518                  print("Speech detected. Listening...")
519
520                  # Start with current chunk
521                  utterance_chunks = [chunk]
522
523                  silence_count = 0
524              continue
```

This is the exact ownership point for both start confirmation and the missing pre-roll. The state transition saves the current chunk only.

A3. Batched waveform passed to transcription

Source: pipeline/main_optimized.py:540-567

```
540              # Combine chunks
541              utterance = np.concatenate(
542                  utterance_chunks,
543                  axis=0,
544              )
545              # Stereo -> mono
546              utterance = utterance.mean(axis=1)
```

Tensor conversion and model call:

```
552              wav = torch.from_numpy(utterance).unsqueeze(0).to(stt_model.d)

562              with torch.inference_mode():
563                  transcription = stt_model(
564                      wav,
565                      LANGUAGE,
566                      DECODING,
567                  )
```

Concatenation retains the captured time sequence, channel averaging produces mono, and unsqueeze adds the model's batch dimension.

Appendix B. Translation and successful conversation turns

B1. Translation adapter routes validated languages

Source: nmt/nmt_client.py:188-206

```
188     def _translate(self, text, source_language, target_language):
189         try:
190             src_code = INDIC_LANGUAGE_CODES[source_language]
191         except KeyError:
192             raise NMTError(
193                 f"unsupported source language: {source_language}"
194             ) from None
195         try:
196             tgt_code = INDIC_LANGUAGE_CODES[target_language]
197         except KeyError:
198             raise NMTError(
199                 f"unsupported target language: {target_language}"
200             ) from None
201 
202         model = self._direction(src_code, tgt_code)
203 
204         # Sentence splitting, SPM tokenisation, batching and detokenisation
205         # all happen inside the engine (see bhashini_engine/engine.py).
206         return model.translate_paragraph(text, src_code, tgt_code)
```

The wrapper validates application language identifiers and delegates all linguistic processing to the engine. It does not call a remote translation endpoint.

B2. Groq history commits only after a usable reply

Source: llm/llm_client.py:30-40

```
30      def send_message(self, text):
31          messages = self.messages + [{"role": "user", "content": text}]
32          response = self.client.chat.completions.create(
33              model=self.model,
34              messages=messages,
35          )
36          content = response.choices[0].message.content if response.choices else None
37          if not content or not content.strip():
38              raise RuntimeError("Groq returned an empty response.")
39          self.messages = messages + [{"role": "assistant", "content": content}]
40          return content
```

The assignment to self.messages occurs after response validation. That is why authentication and empty-response failures preserve the previous history.

B3. Translation failure policy belongs to the pipeline

Source: pipeline/main_optimized.py:267-280

```
267     # Nothing to translate: same language on both ends, or NMT unavailable.
268     if source_language == target_language or translator is None:
269         return text, 0.0
270 
271     start = time.perf_counter()
272     try:
273         translated = translator.translate(text, source_language, target_language)
274     except NMTError as exc:
275         elapsed = time.perf_counter() - start
276         print(f"NMT {source_language}->{target_language} failed: {exc}")
277         print("Skipping translation for this turn; passing text through unchanged.")
278         return text, elapsed
279 
280     return translated, time.perf_counter() - start
```

Returning the original text keeps a turn moving but weakens the expected language contract. The adapter raises an error; the pipeline chooses the fallback.

Appendix C. Device placement, speech output and token limits

C1. Restore the CUDA function even when STT loading fails

Source: pipeline/main_optimized.py:196-206

```
196 _real_cuda_is_available = torch.cuda.is_available
197 if STT_DEVICE == "cpu":
198     torch.cuda.is_available = lambda: False
199
200 try:
201     stt_model = AutoModel.from_pretrained(
202         STT_MODEL_ID,
203         trust_remote_code=True,
204     ).to(STT_DEVICE)
205 finally:
206     torch.cuda.is_available = _real_cuda_is_available
```

This is a targeted startup workaround around custom remote STT code. Restoration is protected by finally, but the temporary change applies to the entire process.

C2. Synthesis uses separate voice and content inputs

Source: pipeline/main_optimized.py:345-364

```
345     with torch.inference_mode(): # Interface mode doesn't do grad calc, backpropagation, etc...
346         generation = tts_model.generate(
347             input_ids=description_input_ids,
348             attention_mask=description_attention_mask,
349             prompt_input_ids=prompt_input_ids,
350             prompt_attention_mask=prompt_attention_mask,
351         )
352     # output of model which is the waveform represented as a pytorch tensor
353
354     # if TTS_DEVICE == "cuda":
355     #     torch.cuda.synchronize()
356
357     # generation_time = time.perf_counter() - start_time
358
359     audio = generation.cpu().numpy().squeeze().astype(np.float32)
360     # .cpu() -> move the tensor to cpu, because after this we use np and sf which doesn't require gpu
361     # .numpy() -> because sd expects a np array
362     # .squeeze() -> if audio is [1,n] or [n,1] squeeze makes them [n] (removes dimension of size 1).
363     # .astype(np.float32) -> the model runs in fp16, but sounddevice.play() only
364     # supports float32/int32/int16/int8/uint8, not float16.
```

The model's generated waveform is converted to the float32 format expected by the playback library.

C3. Translation backend limit

Source: nmt/bhashini_engine/engine.py:209-218

```
209         translations = self.translator.translate_batch(
210             tokenized_sents,
211             max_batch_size=self.max_batch_size,
212             batch_type="examples",
213             max_input_length=160,
214             max_decoding_length=self.max_decoding_length,
215             beam_size=self.beam_size,
216         )
217         translations = [" ".join(x.hypotheses[0]) for x in translations]
218         return translations
```

The explicit 160-token input limit is the reason the report flags a mismatch with preprocessing's 256-token chunks. Beam search and maximum output length affect translation, independently of the source-language/script normalization steps.